On Convergence of Adam With Data Dependent Stepsize

Alokendu Mazumder, Rishab Sabharwal, Bhartendu Kumar, Manan Tayal, *Graduate Student Member, IEEE*, Chirag Garg, Arnab Roy, and Punit Rathore, *Member, IEEE*

***Abstract*—In neural network training, Adam is a widely favored optimization algorithm. A crucial factor in their performance is selecting the correct stepsize, significantly impacting their effectiveness. The stepsize is a crucial hyperparameter that requires careful tuning to achieve optimal results. For training large models on large-scale datasets, tuning the stepsize can be very expensive and often time-consuming. Stepsize schedulers are proposed to navigate the trajectory on highly non-convex losses in deep learning, but many decaying schedulers (such as linear decay, exponential decay, and step decay) can lead to divergence or slow convergence compared to constant stepsizes. This suggests that rapidly decaying rates play a crucial role in driving convergence. Building on this observation, we propose a new estimate of constant stepsize that depends on the dynamics of the network and the data. This approach eliminates two major issues: the problems associated with decaying schedulers and the need for an exhaustive search for stepsizes. Numerical experiments demonstrate that our estimated stepsize closely matches the best-performing stepsizes found via exhaustive search, significantly reducing the need for manual tuning. We also show that under certain conditions, our stepsize estimator performs on par with various decaying schedulers. We further conduct experiments on the full ImageNet dataset to demonstrate the effectiveness of our proposed method in a large-scale setting. Beyond image classification, we also apply our approach to large language model (LLM) fine-tuning, benchmarking it against a range of modern optimizers. These results highlight the versatility of our method across diverse domains. Furthermore, we prove that with our estimated stepsize, Adam can reach critical points for smooth, non-convex objectives within bounded runtime, providing strong theoretical support for our approach. We analyze both deterministic and stochastic versions of Adam and establish sufficient conditions for our proposed stepsize to achieve asymptotic convergence of the gradients to zero. Our results also show that the key driver for Adam's convergence is the use of non-increasing stepsizes, despite accumulating a few past gradients.**

***Impact Statement*—Hyperparameter tuning, particularly stepsize selection, is a critical challenge in training deep learning models, often requiring significant computational resources and time. Our research addresses this by proposing a novel, data- and network-driven constant stepsize approach that eliminates the reliance on decaying schedulers and exhaustive search for optimal stepsizes. This innovation substantially reduces the computational cost of hyperparameter tuning, making it especially beneficial for large-scale models and datasets. From a technological perspective, this advancement enhances the accessibility of deep learning by enabling more efficient training workflows, thereby accelerating research and development across diverse AI applications. Economically, it reduces the resource demands of model training, offering significant cost savings for industries deploying AI at scale. Theoretically, our work contributes to the foundational understanding of optimization in deep learning by establishing conditions under which our stepsize guarantees asymptotic convergence and performs competitively with existing decaying schedulers. By simplifying stepsize selection while maintaining robust convergence properties, our approach has the potential to democratize deep learning research and deployment, fostering innovation across fields such as healthcare, finance, and climate science, where AI-driven solutions are critical. This work bridges the gap between theoretical rigor and practical utility, paving the way for more efficient and scalable AI systems.**

***Index Terms*—Convergence, non-convex optimization, stepsize.**

I. Introduction

OPTIMIZATION problems in machine learning are often structured as minimizing a finite sum $\min_{x\in\mathbb{R}^n} f(x)$, where $f:\mathbb{R}^n \mapsto \mathbb{R}$ and $f(x) = \frac{1}{k}\sum_{i=1}^{k} f_i(x)$. Each $f_i:\mathbb{R}^n \mapsto \mathbb{R}$ typically exhibits non-convex behavior, particularly in neural network domains. A prominent method for tackling such

Received 12 November 2025; accepted 15 January 2026. Date of publication 22 January 2026; date of current version 31 August 2026. This work was supported by the SERB Start-up Research Grant (SRG), Government of India. The work of Alokendu Mazumder was supported by the Kotak AI-ML Center (KIAC) Ph.D. Fellowship and the Prime Minister's Research Fellowship (PMRF), India. The work of Manan Tayal was supported by the PMRF. The work of Punit Rathore was supported by the Arcot Ramachandran Young Investigator Award Grant of the Indian Institute of Science (IISc), Bengaluru, India. This article was recommended for publication by Associate Editor Johan A.K. Suykens upon evaluation of the reviewers' comments. *(Bhartendu Kumar, Manan Tayal, Chirag Garg, and Arnab Roy contributed equally to this work.) (Corresponding author: Alokendu Mazumder.)*

Alokendu Mazumder, Manan Tayal, and Punit Rathore are with Robert Bosch Center for Cyber-Physical Systems (RBCCPS), Indian Institute of Science (IISc), Bengaluru 560012, India (e-mail: alokendum@iisc.ac.in).

Rishab Sabharwal was with Robert Bosch Center for Cyber-Physical Systems (RBCCPS), Indian Institute of Science (IISc), Bengaluru 560012, India and is now with the University of Edinburgh, EH8 9YL Edinburgh, Scotland.

Bhartendu Kumar was with the Indian Institute of Science (IISc), Bengaluru 560012, India and is now with the Microsoft Research India, Bengaluru 560001, India.

Chirag Garg was with Robert Bosch Center for Cyber-Physical Systems (RBCCPS), Indian Institute of Science (IISc), Bengaluru 560012, India and is now with the Department of Computer Science and Engineering, National Institute of Technology, Raipur 492010, India.

Arnab Roy was with Robert Bosch Center for Cyber-Physical Systems (RBCCPS), Indian Institute of Science (IISc), Bengaluru 560012, India and is now with the Department of Computer Science and Automation, Indian Institute of Science (IISc), Bengaluru 560012, India.

This article has supplementary downloadable material available at https://doi.org/10.1109/TAI.2026.3656866, provided by the authors.

Digital Object Identifier 10.1109/TAI.2026.3656866

Algorithm 1: Adam Algorithm Pseudocode [5].

Input: stepsize: $\alpha \in (0,1]$, $\beta_1, \beta_2 \in [0,1)$, initial starting point $\mathbf{w}_0 \in \mathbb{R}^d$, a constant vector $\rho\mathbf{1} > \mathbf{0} \in \mathbb{R}^d$, we assume we have access to a noisy oracle for gradients of $f : \mathbb{R}^d \mapsto \mathbb{R}$

Initialization: $\mathbf{m}_0 = \mathbf{0}$, $\mathbf{v}_0 = \mathbf{0}$

for t *from 1 to* T: **do**

$\mathbf{g}_t = \nabla f(\mathbf{w}_t)$

$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t$

$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2$

$\mathbf{V}_t = \mathtt{diag}(\mathbf{v}_t)$

$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha(\mathbf{V}_t^{1/2} + \mathtt{diag}\,(\rho\mathbf{1}))^{-1}\,\mathbf{m}_t$

End

problems is stochastic gradient descent (SGD), where updates occur iteratively as $x_{t+1} := x_t - \alpha \nabla \tilde{f}_{i_t}(x_t)$, with α denoting the stepsize and $\tilde{f}_{i_t}$ being a randomly chosen function from $\{f_1, f_2, \ldots, f_k\}$ at each iteration t. SGD is preferred for training deep neural networks due to its computational efficiency, especially with mini-batch training, even with large datasets [1].

Adaptive variants of SGD, which incorporate past gradients through averaging, have gained traction due to their ability to track gradient scaling on an individual parameter basis [2]. These methods are preferred for their perceived ease of tuning compared to traditional SGD. Adaptive gradient methods typically update using a vector obtained by applying a linear transformation, often referred to as *diagonal preconditioner* to the linear combination of all previously observed gradients. This preconditioning is believed to enhance algorithm robustness to hyperparameter choices, making them less sensitive to initial settings.

Adagrad [3] showed superior performance, especially in scenarios with small or sparse gradients. However, its effectiveness diminishes in situations with non-convex loss functions and dense gradients due to rapid stepsize decay. To address this, variants such as RMSProp [4], Adam [5] (Algorithm 1), and Adadelta [6] have been proposed. These methods mitigate stepsize decay by employing exponential moving averages of squared past gradients, thereby limiting the reliance on all past gradients during updates. While these algorithms have found success in various applications [7], they have also been observed to exhibit non-convergence in many settings [8], especially in deterministic environments where noise levels are controlled during optimization. This regulation is accomplished either by employing larger batches [9], [10], [11] or by integrating variance-reducing techniques [12], [13].

For the reader's convenience, we first clarify a few necessary notations used in the forthcoming Generic Adam. First, we denote $\mathbf{w} \in \mathbb{R}^d$ as the optimizing variable and $\mathbf{g}_t$ as the stochastic gradient at the t^{th} iteration respectively, $\alpha > 0$ base stepsize, $0 < \rho << 1$ as the stabilizer, and $\beta_1, \beta_2 \in (0,1]$ momentum parameter, respectively. Denote $\mathbf{0} = (0, \ldots, 0)^T \in \mathbb{R}^d$, and $\mathbf{1} = (1, \ldots, 1)^T \in \mathbb{R}^d$. All operations, such as multiplying, dividing, and taking the square root, are executed in the coordinate-wise mode.

Despite their widespread adoption in solving neural network problems, adaptive gradient methods like RMSProp and Adam often lack theoretical justifications in non-convex scenarios, even when dealing with exact or deterministic gradients [14]. Several sufficient conditions (SCs) have been proposed to guarantee the global convergence of Adam, which can be further classified into the following two categories.

A. Problems With Stepsize Decay

The main cause of divergences in Adam and RMSProp predominantly arises from the disparity between the two successive stepsizes [8]

$$\Gamma_t = \frac{\sqrt{\mathbf{v}_t}}{\alpha_t} - \frac{\sqrt{\mathbf{v}_{t-1}}}{\alpha_{t-1}}. \tag{1}$$

If the positive definiteness property of Γ_t is violated, Adam and RMSProp may experience divergence. The $\Gamma_t > 0$ constraint can be relaxed [15], and convergence of Adam can still be achieved when the condition $(\sqrt{\mathbf{v}_t}/\alpha_t) \geq c(\sqrt{\mathbf{v}_{t-1}}/\alpha_{t-1})$ holds for all t and some positive constant c. Prior research efforts [16], [17], [18], [19], [20], [21] have focused on establishing convergence properties of optimization algorithms such as RMSProp and Adam. These investigations typically utilize stepsize schedules of the form $\alpha_t = \frac{\alpha}{t^a}$ for all $t \in \{1, 2, \ldots, T\}$, $\alpha \in (0,1)$, and $a > 0$, or other time-dependent variations like $\alpha_t = \alpha(1-\beta_1)\sqrt{(1-\beta_2^t/1-\beta_2)}$. While a non-increasing (diminishing) stepsize is crucial for convergence, empirical analysis suggests that rapid decay of the stepsize can lead to sub-optimal outcomes, as discussed in the next section.

B. Problems With Temporal Decorrelation

The divergence observed in RMSProp stems from imbalanced stepsizes [8], [22] rather than the absence of $\Gamma_t > 0$. Leveraging this insight, AdaShift [22] was proposed, which incorporates a temporal decorrelation technique to address the inappropriate correlation between $\mathbf{v}_t$ and the current second-order moment $\mathbf{g}_t^2$. This approach requires the adaptive stepsize α_t to be independent of $\mathbf{g}_t^2$. However, it is worth noting that the convergence analysis of AdaShift was primarily limited to RMSProp for resolving the convex counterexample presented in [8].

In contrast to the aforementioned modifications and restrictions, we introduce an alternative SC to ensure the global convergence of the original Adam. The proposed SC (refer to Section IV-A) depends on a constant stepsize α_{ours} (our estimated stepsize) and on the parameter β_1. Our SC does not necessitate rapidly decaying stepsizes and positive definiteness of Γ_t. It is easier to verify and more practical compared to (Section I-B).

NOTE: A recent study [23] proposes a similar convergence rate to ours using a constant stepsize approach for mini-batch Adam, where they employ $\alpha_t = (\alpha/\sqrt{T})$, with α being any positive real number. However, our work asks the questions:

1) For better convergence[1] of Adam with non-decreasing or constant stepsizes, is $\alpha_t = (\alpha/\sqrt{T})$ sufficient?

[1]Better convergence refers to the gradient norm of the loss approaching zero more closely as iterations progress. If two algorithms, $\mathcal{A}_1$ and $\mathcal{A}_2$, drive the gradient norm of the loss to 0.1 and 0.001 respectively, $\mathcal{A}_2$ demonstrates superior convergence. To assess the convergence speed accurately, one must analyze the loss curve plot.

2) Are there any other terms related to the dynamics of the model and data that need to be integrated as constants in the stepsizes proposed above?
3) If such terms exist, will they lead to better convergence?

Our work specifies the *exact* stepsize that ensures convergence with fewer assumptions and contains information about the model dynamics and data distribution in the form of *Lipschitz smoothness of the loss*.

This article is organized as follows. Section II presents basic definitions and pseudocodes that will be used throughout our theoretical analysis. In Section III, we provide the motivation for this work and outline the major contributions of the article. Section IV discusses the convergence guarantees for Adam with a fixed stepsize. In Section V, we present the details of the designed experiments and analyze the results. Finally, Section VII concludes the article.

II. Notations and Definitions

In the following, we denote as $\mathbf{u}^T\mathbf{w}$ and $\|\mathbf{w}\|_2$ the scalar product and L_2-norm of the Hilbert space $\mathbb{R}^n$ respectively. For any differentiable and real-valued function $f:\mathbb{R}^n\to\mathbb{R}$ and any point $\mathbf{w}_0\in\mathbb{R}^n$, we will denote as $\nabla_{\mathbf{w}}f(\mathbf{w}_0)\in\mathbb{R}^n$ the gradient of f at $\mathbf{w}_0$. The double derivative of f at $\mathbf{w}_0$ is denoted by $\nabla^2_{\mathbf{w}}f(\mathbf{w}_0)\in\mathbb{R}^{n\times n}$ which is a square matrix, also called the *Hessian matrix*. Next, we define the Lipschitz property, a standard assumption in optimization literature, which we assume in all our proofs for non-convex objectives. Additionally, this section presents the pseudocode for the Adam optimizer.

Definition 1 (K - Smoothness): If $f:\mathbb{R}^d\to\mathbb{R}$ is at least once differentiable, then we call it K smooth for some constant $K>0$, if $\forall\mathbf{w}_1,\mathbf{w}_2\in\mathbb{R}^n$, the following inequality holds

$$f(\mathbf{w}_2)\leq f(\mathbf{w}_1)+\nabla_{\mathbf{w}}f(\mathbf{w}_1)^T(\mathbf{w}_1-\mathbf{w}_2)+\frac{K}{2}||\mathbf{w}_1-\mathbf{w}_2||_2^2.$$

If a function $f:\mathbb{R}^n\to\mathbb{R}$ is K-smooth, then K is essentially the Lipschitz constant of the gradient of f. Henceforth, we will refer to K as the Lipschitz smoothness of the function, which is the same as the Lipschitz constant of the gradient of the function. Alternatively, we can define K-smoothness as:

Definition 2 (K - Smoothness): A function $f:\mathbb{R}^d\to\mathbb{R}$ is called Lipschitz smooth if there exists a constant $K>0$ such that

$$\forall\mathbf{w}_1,\mathbf{w}_2\in\mathbb{R}^n,\|\nabla_{\mathbf{w}}f(\mathbf{w}_1)-\nabla_{\mathbf{w}}f(\mathbf{w}_2)\|_2\leq K\|\mathbf{w}_1-\mathbf{w}_2\|_2.$$

The smallest K for which the previous inequality is true is called the Lipschitz constant of ∇f or Lipschitz smoothness of f and will be denoted $K(f)$. Note, from here onwards K and $K(f)$ can be used interchangeably, denoting the function's Lipschitz smoothness. An approximated Lipschitz smoothness constant is denoted by $\hat{K}$.

III. Motivation Contributions

In practical scenarios, such as learning latent variables from vast datasets with unknown distributions, the goal is to solve the optimization problem

$$\min_{\mathbf{w}\in\mathbb{R}^d}\mathcal{L}(\mathbf{w}):=\mathbb{E}_{\zeta\sim\mathcal{P}}[l(\zeta,\mathbf{w})]. \tag{2}$$

Here $\mathcal{L}:\mathbb{R}^n\to\mathbb{R}$ is a non-convex loss function, ζ is a random variable with an unknown distribution $\mathcal{P}$ and $l(\zeta,\mathbf{w})$ quantifies the performance of parameter vector $\mathbf{w}\in\mathbb{R}^n$ on the random variable ζ. Common iterative optimization algorithms like Adam and RMSProp are often employed to solve this problem. It is observed that using a more aggressive constant stepsize often leads to favorable outcomes [24], [25].

A. Reducing Exhaustive Search for Stepsize

Determining the optimal stepsize typically requires an exhaustive search. Specifically, when optimizing a parameter set $\mathbf{w}\in\mathbb{R}^d$, the goal is to find a parameter $\mathbf{w}^*$ that aligns with a minimum in the loss landscape $\mathcal{L}(\mathbf{w})$. We propose a simple method to estimate a constant stepsize close to the best-performing stepsize across a wide range of acceptable stepsizes. With our proposed stepsize, one can achieve optimal results with minor tuning, rather than conducting a full exhaustive search over all acceptable stepsizes.

B. Why Not Use Decaying Schedulers?

Gradient descent-based algorithms, including those with momentum, iteratively update parameters using $\mathbf{w}_{t+1}=\mathbf{w}_t-\alpha_t g(\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}_t))$, where α_t is the stepsize at iteration t and $g:\mathbb{R}^d\mapsto\mathbb{R}^d$ is a function incorporating the gradient and momentum terms. Convergence of the parameter sequence $\{\mathbf{w}_0,\mathbf{w}_1,\ldots,\mathbf{w}_{T-1}\}$ to $\mathbf{w}^*$ necessitates the sequence $\{(\mathbf{w}_{t+1}-\mathbf{w}_t)\}$ to converge to 0 as $t\to T$. This corresponds to the sequence $\{\alpha_t g(\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}_t))\}$ converging to $0\in\mathbb{R}^d$. The literature often opts for a non-increasing stepsize α_t, such as $\alpha_t\propto\frac{1}{t^a}$, where $a>0$, to theoretically prove convergence. Despite non-zero gradient norms, α_t can cause $\{\alpha_t g(\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}_t))\}$ to approach 0, even if $\{\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}_t)\}$ does not. Although accumulating past gradients helps mitigate the decay of the stepsize, there is a risk that stepsizes with rapid decay may dominate. Our empirical analysis in Section V demonstrates that Adam, with non-increasing stepsizes, does not aggressively drive the gradient norm of the loss towards zero. With a fixed stepsize $\alpha_t=\alpha>0$, if $\mathbf{w}_t\to\mathbf{w}^*$, then $\|\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}_t)\|_2\to 0$. This ensures convergence to a saddle point of $\mathcal{L}(\mathbf{w})$ with a fixed stepsize, an assurance not available with a non-increasing and iteration-dependent stepsize. Additionally, as mentioned in Section I-A, decaying stepsizes suffer from the problem of divergence under certain conditions.

This research aims to derive an exact constant stepsize that eliminates the need for an exhaustive search over possible stepsizes and avoids the issues associated with decaying stepsize schedulers. Our proposed stepsize serves as a reliable starting point, being very close to the best-performing stepsize found through extensive search. We also ensure convergence in both deterministic and stochastic Adam iterations beyond a time threshold $t\geq T(\beta_1,\rho)$ for our proposed stepsize. Here, $T(\beta_1,\rho)$ is a natural number.

C. Summary of Our Contributions

The major contributions of this article are summarized as follows.

1) We derive an exact constant stepsize $\alpha_{\text{ours}} = \sqrt{(2\mathcal{L}(w_0)/\hat{K}.T)}$ that eliminates the need for an exhaustive search over possible stepsizes. Our proposed stepsize provides an excellent starting point, as it is very close to the best-performing stepsize identified through an exhaustive search. Full analysis is presented in Section IV.
2) We perform extensive experiments and show that under certain conditions our proposed stepsize can be readily used as a drop-in-replacement for various decaying schedulers, eliminating the time required to tune the schedulers.
3) The convergence of deterministic as well as stochastic Adam is guaranteed by our proposed stepsize. To the best of our knowledge, this study is the first to theoretically guarantee Adam's global convergence (gradient norm of loss function converges to 0) with an exact constant stepsize, which depends on the dynamics of the model and data.
4) We introduced a simple method to estimate the Lipschitz smoothness[2] of the loss function *w.r.t* the network parameters. We offer a probabilistic guarantee for the convergence of our estimated Lipschitz constant to its true value.
5) We empirically demonstrate that the proposed stepsize is competitive with a wide range of stepsize schedulers used with Adam, as well as with several modern adaptive optimizers, across both vision and LLM based tasks, including large-scale experiments. Section V contains all the experiments.

IV. Proposed Stepsize Estimate for Training With Adam

Previously, it has been shown that deterministic RMSProp and Adam can converge under certain conditions with adaptive stepsize [8], [20], [26]. Here, with our estimated stepsize, we give the first result about convergence to criticality for deterministic and stochastic Adam with constant stepsize, albeit under a certain technical assumption about the loss function (and hence on the noisy first-order oracle). *The proofs of the theorems discussed in this section, along with additional experiments, are provided in the Supplementary Material.*

A. SC for Convergence of Adam

Below are the commonly employed assumptions for analyzing the convergence of stochastic algorithms for non-convex problems:

1) The minimum value of the problem $\mathcal{L}^* = argmin_{\mathbf{w}\in\mathbb{R}^d}\mathcal{L}(\mathbf{w})$ is lower bounded, *i.e.* $\mathcal{L}^* > -\infty$.
2) The stochastic gradient $\mathbf{g}_t$ of the loss function is an unbiased estimate, *i.e.* $\mathbb{E}[\mathbf{g}_t] = \nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}_t)$.
3) The second-order moment of stochastic gradient $\mathbf{g}_t$ is uniformly upper-bounded, *i.e.* $(\mathbb{E}[\|\mathbf{g}_t\|_2])^2 \leq \mathbb{E}[\|\mathbf{g}_t\|_2^2] \leq \gamma^2$. (Variance inequality[3])
4) For the deterministic version of Adam, we assume bounded gradients, i.e. $\|\nabla f(x_t)\|_2 \leq \gamma$.

In addition, we suppose that the parameters β_1 and stepsize α satisfy the following restrictions.

1) The parameter β_1 satisfies $\beta_1 < \epsilon/(\epsilon+\gamma)$ for some $\epsilon > 0$.
2) Our proposed stepsize $\alpha_t = \alpha \in \mathbb{R}^+$ (the proposed stepsize is mentioned clearly in the below theorems) remains constant throughout the analysis.

Theorem 1 (Deterministic Adam converges with proper choice of constant stepsize): Let the loss function $\mathcal{L}:\mathbb{R}^n \to \mathbb{R}$ be $K-$smooth and let $\gamma < \infty$ be an upper bound on the norm of the gradient of $\mathcal{L}$. Also assume that $\mathcal{L}$ has a well-defined minimiser $\mathbf{w}^*$ such that $\mathbf{w}^* = argmin_{\mathbf{w}\in\mathbb{R}^d}\ \mathcal{L}(\mathbf{w})$. Then the following holds for Algorithm (1):

For any $\epsilon > 0$ and $\rho > (\beta_1\gamma^2/-\beta_1\gamma + \epsilon(1-\beta_1))$ if we let $\alpha = \sqrt{2(\mathcal{L}(\mathbf{w}_0) - \mathcal{L}(\mathbf{w}^*))/KT}$, then there exists a natural number T (depends on β_1 and ρ) such that $\|\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}_t)\|_2 \leq \epsilon$ for some $t \leq T$, where $\delta^2 = (\gamma^2/\rho^2)$.

Theorem 2 (Stochastic Adam converges with proper choice of constant stepsize): Let the loss function $\mathcal{L}:\mathbb{R}^n \mapsto \mathbb{R}$ be $K-$smooth and be of the form $\mathcal{L} = \sum_{j=1}^m \mathcal{L}_j$ such that (a) each $\mathcal{L}_j$ is at least once differentiable, (b) the gradients satisfy $sign(\mathcal{L}_r(\mathbf{w})) = sign(\mathcal{L}_s(\mathbf{w}))$ for all $r, s \in \{1, 2, \ldots, m\}$, (c) $\mathcal{L}$ has a well-defined minimiser $\mathbf{w}^*$ such that $\mathbf{w}^* = argmin_{\mathbf{w}\in\mathbb{R}^d}$ $\mathcal{L}(\mathbf{w})$. Let the gradient oracle, upon invocation at $\mathbf{w}_t \in \mathbb{R}^d$, randomly selects j_t from the set $\{1, 2, \ldots, m\}$ uniformly, and then provides $\nabla\mathcal{L}_{j_t}(x_t) = \mathbf{g}_t$ as the result.

Then, for any $\epsilon > 0$ and $\rho > (\beta_1\gamma^2/-\beta_1\gamma + \epsilon(1-\beta_1))$ if we let $\alpha = \sqrt{2(\mathcal{L}(\mathbf{w}_0) - \mathcal{L}(\mathbf{w}^*))/KT}$, then there exists a natural number T (depends on β_1 and ρ) such that $\mathbb{E}[\|\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}_t)\|_2] \leq \epsilon$ for some $t \leq T$, where $\delta^2 = \frac{\gamma^2}{\rho^2}$.

Remark 1: With our analysis, we showed that both deterministic and stochastic Adam with our proposed constant stepsize converges with rate $\mathcal{O}\left(1/T^{1/4}\right)$. This convergence rate is similar to the convergence rate of SGD proposed in [27]. Our motivation behind these theorems was primarily to understand the conditions under which Adam can converge, especially considering the negative results presented in [8]. However, we acknowledge that it is still an open problem to tighten the analysis of deterministic as well as stochastic Adam and obtain faster convergence rates than we have shown in the theorem. One thing to keep in mind is that our convergence rate is obtained for a fixed stepsize, and having a decaying stepsize provides additional opportunities for analysis to tighten the bound [16], [17], [18], [19], [20], [21]. But in practical settings, these decaying schedulers have problems mentioned in Section I-A and Section B.

Remark 2: Based on the preceding two theorems, it becomes evident that the Adam achieves convergence when equipped with an appropriately chosen constant stepsize $\alpha = \sqrt{2(\mathcal{L}(\mathbf{w}_0) - \mathcal{L}(\mathbf{w}^*))/KT}$, *according to our analysis*. Subsequently, we analyze the given stepsize and conduct extensive experiments to demonstrate that this stepsize serves as a good starting point for training models. Our proposed stepsize performs comparably to the best-performing stepsize identified through an exhaustive search, eliminating the need

[2]Our derived stepsize, crucial for ensuring convergence, is determined by the Lipschitz smoothness of the loss function.

[3]For any random variable $\mathbf{X}$, $\text{Var}(\mathbf{X}) \geq 0 \implies (\mathbb{E}[\mathbf{X}])^2 \leq \mathbb{E}[\mathbf{X}^2]$.

for tuning the stepsize. Our stepsize also reduces the gradient norm effectively and more aggressively tends towards zero compared to other stepsize scheduling techniques (listed in Table II in supplementary material). Hence, our empirical investigations validate that the chosen stepsize ensures convergence in practice. We compare our stepsize with several state-of-the-art schedulers and an array of constant stepsizes in Section V.

B. Analysis of Our Estimated Stepsize

The optimal stepsize, as per Theorems 1 and 2, depends on factors like the Lipschitz smoothness of the loss[4] (K), initial and final loss values and the total number of iterations/epochs (T). All our experiments are conducted on image classification tasks using a wide variety of neural network architectures, with cross-entropy loss employed, which has a minimum value of 0. Hence, we set the final loss $\mathcal{L}(\mathbf{w}^*)$ to zero. Thus, our approximate stepsize is now expressed as $\alpha_{\text{ours}} \approx \sqrt{(2\mathcal{L}(\mathbf{w}_0)/\hat{K}T)}$, making it practical while still capturing the core concept of Theorems 1 and 2. Here $\hat{K}$ is the approximated smoothness of the loss, defined in the next section.

C. Approximating the Lipschitz Smoothness of Loss

Determining the appropriate Lipschitz smoothness for a neural network is a key area of research in deep learning, with various methods proposed for estimation, as demonstrated in recent works [28], [29], [30], [31], [32]. Estimating the stepsize prior to training requires estimation of the Lipschitz smoothness of the loss function. For locally Lipschitz smooth functions (i.e., functions whose restriction to some neighborhood around any point is Lipschitz), the Lipschitz smoothness may be computed using its Hessian matrix.

Theorem 3 (Rademacher [33] [22, Theorem 3.1.6]): If $f : \mathbb{R}^d \mapsto \mathbb{R}$ is a locally Lipschitz smooth function, then f is differentiable almost everywhere. Moreover, if f is at least twice differentiable and $\nabla_{\mathbf{w}} f$ is Lipschitz continuous, then

$$K(f) = sup_{\mathbf{w}\in\mathbb{R}^d} \|\nabla^2_{\mathbf{w}} f\|_2 \tag{3}$$

where $\|\mathbf{A}\|_2$ is the spectral norm of the matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$.

For large networks, directly optimizing according to Theorem 3 can be time-consuming. Instead, we propose a novel technique to approximate the Lipschitz smoothness. It is important to note that this work does not aim to estimate the best Lipschitz smoothness for the loss or network; rather, it focuses on finding the best stepsize to avoid extensive hyperparameter searches. We introduce Algorithm 2 to find a conservative approximation of the Lipschitz smoothness of the loss,[5] supported by mathematical proof that our estimate *converges in distribution* to the original Lipschitz smoothness asymptotically. We shall now prove that our estimated Lipschitz smoothness $(\hat{K})$ converges to the real Lipschitz smoothness (K) asymptotically in distribution.

[4]The Lipschitz smoothness of the loss is with respect to network parameters.

[5]Algorithm 2 is given for the full-batch scenario. The algorithm for the mini-batch case is deferred to the supplementary material due to space constraints.

Algorithm 2: Estimating A Conservative Lipschitz Smoothness of Loss Function.

Input: Dataset: $\mathcal{X} \sim \mathcal{P}$, Loss function: $\mathcal{L} : \mathbb{R}^d \to \mathbb{R}$, A network parameterized by $\mathbf{w} \in \mathbb{R}^d \sim \mathcal{W}$, No. of iterations: N
Output: $\hat{K}$
Initialization: $\hat{K} = 0$, Iteration Number $n = 0$
for *n from 1 to N:* **do**
  Randomly sample the network weights: $\mathbf{w}_n \overset{\text{random}}{\sim} \mathcal{W}$
  Compute loss for the current pass: $\mathcal{L}(\mathbf{w}_n) = \mathbb{E}_{\zeta\sim\mathcal{P}}[l(\zeta, \mathbf{w}_n)]$
  Compute the spectral norm of hessian at the current instant using power method [34]: $\|\nabla^2_{\mathbf{w}}\mathcal{L}(\mathbf{w}_n)\|_2$
  Estimate the Lipschitz smoothness $(\hat{K})$: $\hat{K} = max(\|\nabla^2_{\mathbf{w}}\mathcal{L}(\mathbf{w}_n)\|_2, \hat{K})$
End

Theorem 4: Given $\|.\|_2$ and $\nabla^2_{\mathbf{w}}\mathcal{L}$ are continuous functions and $\|\nabla^2_{\mathbf{w}}\mathcal{L}\|_2$ is bounded from above. If $\mathcal{L}$ is locally Lipschitz smooth then, $K(\mathcal{L}) = sup_{\mathbf{w}\in\mathbb{R}^d}\|\nabla^2_{\mathbf{w}}\mathcal{L}\|_2$. Let $\hat{K}$ be a random variable defined as $\hat{K} = \underset{1\le j\le N}{max}\|\nabla^2_{\mathbf{w}}\mathcal{L}(\mathbf{w}_j)\|_2$, where $\mathbf{w}_j$, $j \in \{1, 2, \ldots, N\}$ are i.i.d. samples drawn from a full domain distribution $\mathcal{W}$. Then, as $N \to \infty$, the random variable $\hat{K}$ converges in distribution to $K(\mathcal{L})$.

Mathematically

$$\lim_{N\to\infty} P(\hat{K} \le k) = \begin{cases} 0 & \text{if } k < K(\mathcal{L}) \\ 1 & \text{if } k \ge K(\mathcal{L}). \end{cases} \tag{4}$$

D. Analysis of Estimated Lipschitz Smoothness

Algorithm 2, proposed for estimating the Lipschitz smoothness/constant of the loss, assumes that the probability distribution from which weights are sampled must have a full domain. Under this assumption, the algorithm converges in distribution to the true Lipschitz constant of the network (see Theorem 4). However, in practice, sampling from regions where the probability density function is very low (close to zero) is not feasible. As a result, Algorithm 2 typically yields a conservative lower bound on the true Lipschitz smoothness. While asymptotically, under ideal conditions, the algorithm can estimate the true global Lipschitz constant, practical limitations such as the inability to sample extreme values from distributions like the standard normal mean that Algorithm 2 often estimates a conservative local Lipschitz constant.

This conservative estimation selects only the *good* weights and implicitly *rejects* the bad weights (weights where the loss surface has sharp minima). Detailed discussion is given in the next section.

E. Connection of Our Proposed Lipschitz Estimation With Generalization

When sampling points from a standard normal distribution and using Algorithm 2 to estimate the Lipschitz smoothness of the loss, we implicitly ensure that the model's weights are drawn from a standard normal distribution during training. This approach effectively computes the Lipschitz constant for an L_2 regularized loss, consistent with the maximum A posteriori (MAP) estimation criteria. Consequently, this process promotes a weight space less prone to poor generalization, estimating

a consistent and conservative Lipschitz smoothness for our proposed stepsize, free from the effect of sharp minima, ensuring the smoothness value does not blow up and results in a vanishing stepsize.

From Theorem 4, $\hat{K} \xrightarrow{d} K(\mathcal{L})$.[6] Optimization of the loss gradient can be achieved through iterative techniques like steepest descent [35], backtracking [35], and Wolfe's and Goldstein's conditions [36], aiming to identify the direction (weights $\mathbf{w}$) maximizing the gradient norm. However, non-convex loss surfaces in deep learning may lead to convergence to sub-optimal local maxima. Also, losses with unbounded gradients can lead to excessively high Lipschitz constants. Our approach addresses this by leveraging the *convergence in distribution* phenomenon from probability theory. We randomly select directions from a distribution $\mathcal{W}$ and take the maximum gradient norm across all evaluated samples, theoretically converging to the true Lipschitz constant in distribution. In all our experiments, we used the Kaiming He uniform distribution [37] to estimate Lipschitz smoothness because its parameters depend on the deployed network's structure. Ablation studies using various distributions are provided in Section C-G of the supplementary material. We observed that the performance curves (gradient norm/accuracy versus iterations) exhibit similar trends across different distributions.

V. Experimental Setup

We assess Adam's performance with our selected stepsize through experiments on fully connected networks with ReLU activations, a variety of famous convolutional neural networks (CNNs) like LeNet [38],[7] VGG-9 [39], ResNet-18 [40], MobileNet-V2 [41] and transformer-based models like ViT-Base [42]. We extend our analysis to classification tasks on the MNIST, CIFAR-10, CIFAR-100,[8] and ImageNet-100 datasets, using all the model architectures described above. Section C-H of the supplementary material provides a list of the 100 classes selected from the ImageNet-1K dataset [43], which constitute our ImageNet-100 subset. To maintain strict experimental control, we ensure that all network layers have identical widths denoted as h. For experiments, in Sections V-B and C, we used Kaiming He uniform distribution [37] for initialization. Default PyTorch parameters are chosen [https://pytorch.org/docs/stable/nn.init.html]. Furthermore, to demonstrate the effectiveness of our proposed method in large-scale settings and tasks beyond image classification, we conducted experiments on the full ImageNet dataset (see Section V-E) and on large language models (LLMs) (see Section V-F). Before delving into further details about the experiments, we will first describe the model architectures utilized for experiments in the subsequent sections. Lipschitz constant is estimated using **Algorithm 2**.

[6]Theorem 4 holds for mini-batch case too. Refer to supplementary material for more insights.

[7]Experiments on LeNet with MNIST data are deferred to supplementary material due to space constraints.

[8]Experiments on CIFAR-100 are deferred to the supplementary material due to space constraints.

A. Outline of the Experiments Conducted

We conducted classification experiments on the MNIST dataset for various network sizes, encompassing different values of network depth and h. ImageNet-100 experiments are conducted on VGG-9, ResNet-18, MobileNet-V2, and ViT-Base. All experiments are implemented using PyTorch. We compare the performance of Adam using *our proposed stepsize* with:

1) A list of commonly used schedulers and several non-increasing stepsizes that were utilized to demonstrate theoretical convergence for Adam in past literature. The schedulers include: a) linear LR; b) step LR; c) square root decay; d) inverse time decay; and e) exponential decay. The full list with hyperparameters are given in Table II, supplementary material.
2) An array of constant stepsizes $\{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}\}$ which are widely used in literature. Additionally, we conducted a fine search around our proposed stepsize α_{ours} by varying it by a factor of 2. The final comparison is made using the list $\{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}, (\alpha_{\text{ours}}/2), 2\alpha_{\text{ours}}\}$.
3) Modern adaptive optimizers combined with cosine decay schedulers, such as AdaBound [19], Yogi [44], MSVAG [45], RAdam [46], and AdamW [47] in full ImageNet and LLM based experiments.

1) Why $2 \times \alpha_{ours}$ and $\alpha_{ours}/2$ are chosen?: To evaluate the robustness and practical sensitivity of our analytically derived stepsize α_{ours}, we additionally test two scaled variants, $2 \times \alpha_{\text{ours}}$ and $\alpha_{\text{ours}}/2$. This choice is deliberate and motivated by the fact that, in practice, the quantities used to compute α_{ours}, namely the estimated Lipschitz constant $\hat{K}$, the initial loss $\mathcal{L}(w_0)$, and the training horizon T, can vary across architectures and data regimes. Estimation errors in these quantities can lead to slightly over or under aggressive stepsizes. Scaling by a factor of two thus provides a simple yet meaningful way to probe both conservative and aggressive learning behaviors without performing an exhaustive grid search. Specifically, the $2 \times \alpha_{\text{ours}}$ variant explores whether the method remains stable under a more aggressive learning regime, while the $\alpha_{\text{ours}}/2$ variant examines its robustness in a more conservative, noise tolerant setting. Such multiplicative perturbations are standard in optimizer sensitivity analyses and help demonstrate that the proposed formulation yields stepsizes that are well calibrated across diverse architectures, datasets, and optimization landscapes.

Each experiment was executed for 100 epochs. The schedulers are generally in the form of $\alpha(t) = \alpha_0 s(t)$ where $\alpha_0 \in (0, 1]$ and $s(t)$ is a decreasing function. We tune α_0 to get the best baseline results. Below is the list of experiments we have conducted.

1) *Full-Batch Toy Experiments on Fully Connected Networks (Section V-B):* To efficiently benchmark our proposed stepsize in a full batch setting, we constructed reduced versions of standard datasets: mini-MNIST, mini-CIFAR-10, and mini-CIFAR-100. Mini-MNIST includes 5500 training and 1000 test images, while the

mini-CIFAR variants contain 500 training and 100 test images per class (approximately 10% of the originals). These subsets retain the statistical characteristics of their parent datasets, enabling efficient experimentation. We trained multiple fully connected networks on these datasets to compare the performance of our stepsize against various existing stepsizes and schedulers during both training and evaluation.

2) *Mini-Batch Toy Experiments on Fully Connected Networks (Section V-C):* To determine if our findings from the full-batch experiments apply to mini-batch scenarios as described in Theorem 2, we conducted similar experiments using a mini-batch setup with a fixed batch size of 5000 for MNIST, CIFAR-10, and CIFAR-100. We utilized the entire training sets for these experiments. We also empirically justified why such a large batch size is needed for empirical convergence. We conducted a series of experiments using various fully connected network configurations.
3) *Experiments on CNN and Vision Transformers (ViT) (Section V-D):* To evaluate whether our proposed stepsize can perform well on different architectures beyond fully connected networks, we conducted experiments on large CNNs and ViT using challenging datasets such as ImageNet-100, CIFAR-100, and CIFAR-10. We benchmarked the performance of our proposed stepsize across various CNN architectures. Specifically, we chose LeNet (5-layer CNN), VGG-9 (9-layer CNN), ResNet-18 (18-layer CNN), MobileNet-V2 (53-layer CNN), and ViT-Base for our experiments. ImageNet-100 experiments are carried out on VGG-9, ResNet-18, MobileNet-V2, and ViT-Base.
4) *Largescale Experiments on Full ImageNet (Section V-E):* To evaluate the effectiveness of the proposed stepsize strategy in a large-scale setting, we conducted image classification experiments on the full ImageNet dataset using ResNet-18, a standard benchmark architecture in the literature. A batchsize of 256 is taken, and the model is trained for 100 epochs.
5) *Experiments on Large Language Model (Section V-F):* To assess the versatility of our proposed stepsize strategy beyond image classification, we fine-tuned two instruction-tuned LLMs, LLaMA-3.2-3B-Instruct [48], and Qwen-3-4B-Instruct [49], primarily focusing on question answering (generative task) using the SQuAD [50] dataset, with additional evaluation on AG News classification. The batchsize is set to 16 for both datasets, and the total number of finetuning steps is set to 3125.
6) *Effect of Varying the Training Horizon (T) (Supplementary Material Section C-C):* Our proposed stepsize is dependent on the training horizon (T). We provide empirical evidence showing that tuning the training horizon preserves the optimization superiority of our fixed stepsize. Since the training horizon is a critical hyperparameter in practical applications [51], understanding its impact is essential for the success of any new optimization methods. All experiments and findings of this section are deferred to supplementary material.
7) *Effect of Varying ρ (Supplementary Material Section C-D):* The parameter ρ is a small positive value, serving as a hyperparameter in the Adam optimization algorithm. It is introduced to prevent the gradient accumulation term from becoming excessively large, ensuring numerical stability during training. In our experiments, we fine-tune this parameter to evaluate its impact on the performance of our proposed stepsize. All experiments and findings of this section are deferred to supplementary material.
8) *Effect of Initialization on Our Stepsize (Supplementary Material Section C-G):* As our stepsize $\alpha_{\text{ours}} = \sqrt{(2\mathcal{L}(\mathbf{w}_0)/\hat{K}T)}$ depends on the initial loss value and the estimated Lipschitz constant, we conducted experiments to demonstrate the independence of our stepsize with respect to different network initialization techniques. All experiments and findings of this section are deferred to supplementary material.

B. Full-Batch Toy Experiments on Fully Connected Networks

In Fig. 1, we illustrate the variations in gradient norm, training loss, and test loss across iterations for different schedulers in a full-batch setting. These experiments were conducted on a three-layer classifier with 1000 nodes in each hidden layer, trained on mini-MNIST. Additional comparisons for various neural network architectures with different widths and depths are provided in the supplementary material, where similar qualitative trends can be observed. *1) Comparison against various fixed stepsizes*: we tested our stepsize against several other fixed stepsizes, $\{10^{-1}, 10^{-2}, 10^{-3}, 10^{-4}, 10^{-5}, 2 \times \alpha_{\text{ours}}, (\alpha_{\text{ours}}/2)\}$. We wanted to demonstrate that instead of searching extensively for the best stepsize to train a model, one could use our stepsize and achieve better results. All the gradient norm and test accuracy plots are deferred to Section C-5 of the supplementary material due to space constraints. However, those figures clearly demonstrate that directly using our proposed stepsize achieves accuracy on par with other stepsizes, thereby reducing the time required for stepsize tuning. *2) Comparison against various stepsize schedulers*: we compared our stepsize against several well-known schedulers listed in Table II in supplementary material. Our goal was to evaluate their efficacy in driving the gradient norm of the loss function towards zero and observe the training trajectory. We also observed that our proposed stepsize performs better on par with the best-performing schedulers. Looking at Fig. 1, it is clear that our suggested stepsize helps decrease the gradient norm of loss more aggressively towards zero as compared to other schedulers, supporting our *Motivation* in Section III. Even with the accumulation of a few past gradients, deterministic Adam proves ineffective in mitigating the rapid decay of the stepsize. This is evident from Fig. 1, where one can observe a notable difference in gradient norms after 100 epochs between the best-performing scheduler (Exponential in this case) and our stepsize is in order of 10^{-2}.

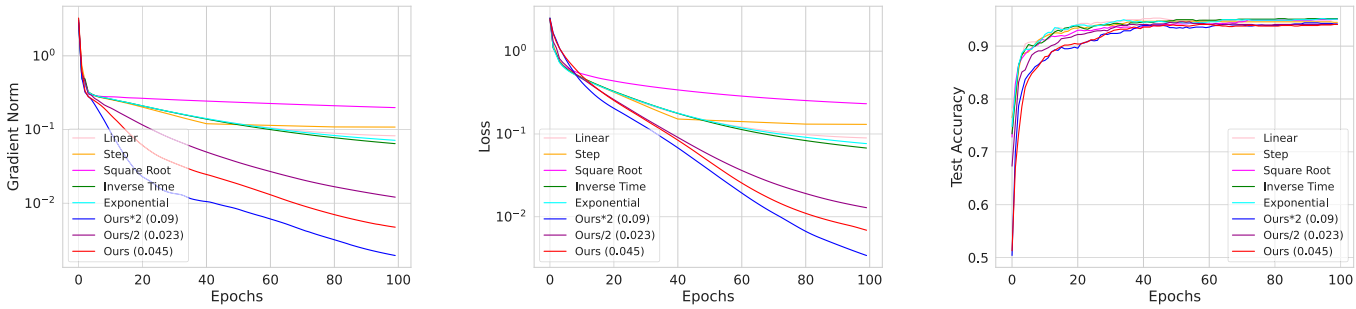


Fig. 1. *Comparison with schedulers: Full-batch* experiments on a 3-layer network with 10^3 nodes/layer, trained on MNIST. Left: Grad. norm versus epochs, Middle: Training loss versus epochs, Right: Test acc. versus epochs.

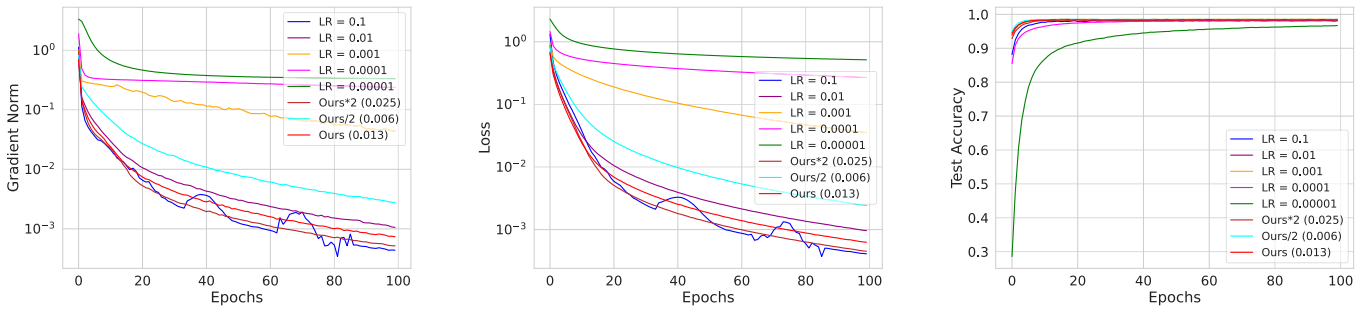


Fig. 2. *Comparison with constant stepsizes: Mini-batch* experiments on a 3-layer network with 10^3 nodes/layer, trained on MNIST. Left: Grad. norm versus epochs, Middle: Training loss versus epochs, Right: Test acc. versus epochs.

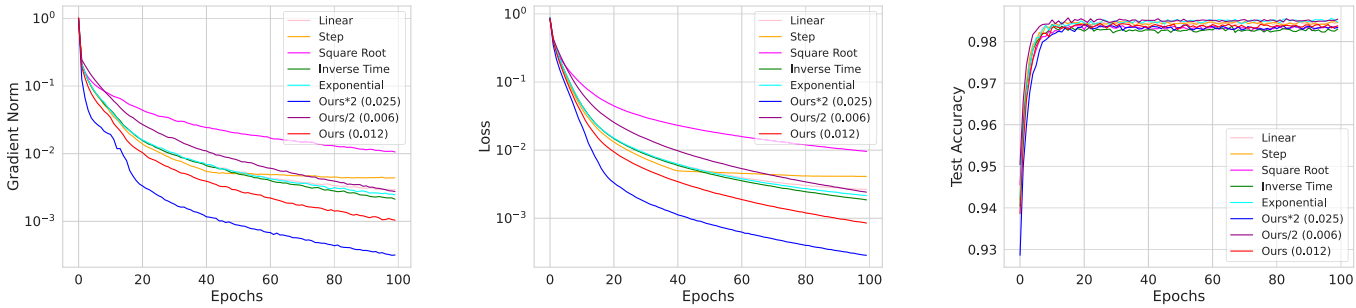


Fig. 3. *Comparison with schedulers: Mini-batch* experiments on a 3-layer network with 10^3 nodes/layer, trained on MNIST. Left: Grad. norm versus epochs, Middle: Training loss versus epochs, Right: Test acc. versus epochs.

Additionally, we observe that the constant stepsize of 10^{-1} shows noticeable jerks during training. This pattern is also evident in other figures, such as Figs. 4–9. This is because, in CNNs, which are more complex models than small fully connected networks, finding local minima is more challenging with such a large stepsize. The stepsize of 10^{-1} is relatively high for training neural networks, which can lead to overshooting of local minima or maxima. In contrast, our stepsize smoothly decreases both the loss and gradient norm during training. This is because our proposed stepsize depends on the neural network dynamics and data (initial loss and Lipschitz constant of network).

Additionally, these figures demonstrate that our stepsize not only reaches any stationary point but tends to approach a local minimum or its neighborhood where the model achieves better or on par as compared to other stepsizes and schedulers test accuracy.[9] The plot of loss versus epoch indicates that our proposed stepsize leads to faster convergence in practice as compared to other schedulers. However, it is worth noting that achieving faster convergence in general is still a challenge. Nevertheless, using this stepsize results in quicker convergence than other schedulers and a range of fixed stepsizes.

C. Comparing Performances in Mini-Batch Setting

In this section, we replicate the same set of experiments in mini-batch setting. We choose 5000[10] as batch size for both the CIFAR-10 and MNIST datasets. From Figs. 2 and 3, we

[9]The test accuracy on the y-axis is *NOT* in percentage. To convert it to a percentage, multiply the y-axis values by 100.

[10]The rationale for the large stepsize is discussed in supplementary material C-I; it is used only in toy experiments, while large-scale experiments employ standard batch sizes consistent with common practice.

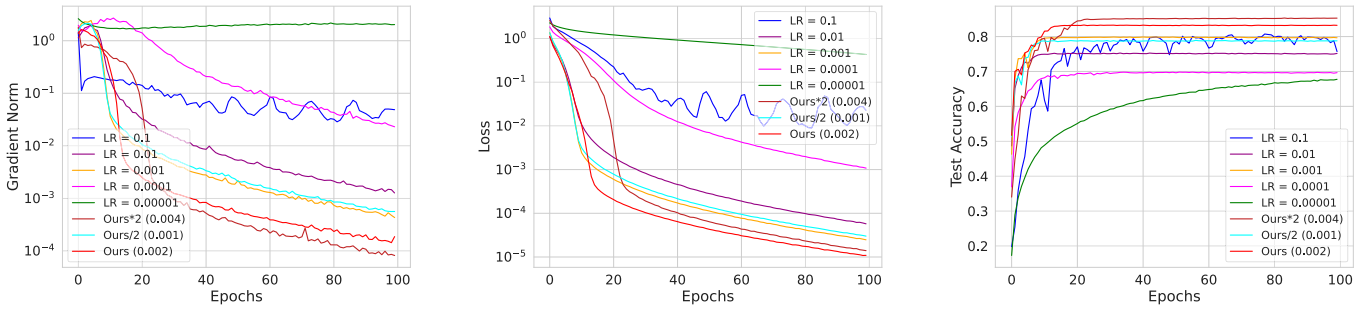


Fig. 4. *Comparison with constant stepsizes: Mini-batch* experiments on VGG-9 architecture with CIFAR-10 data. Left: Grad. norm versus epochs, Middle: Training loss versus epochs, Right: Test acc. versus epochs.

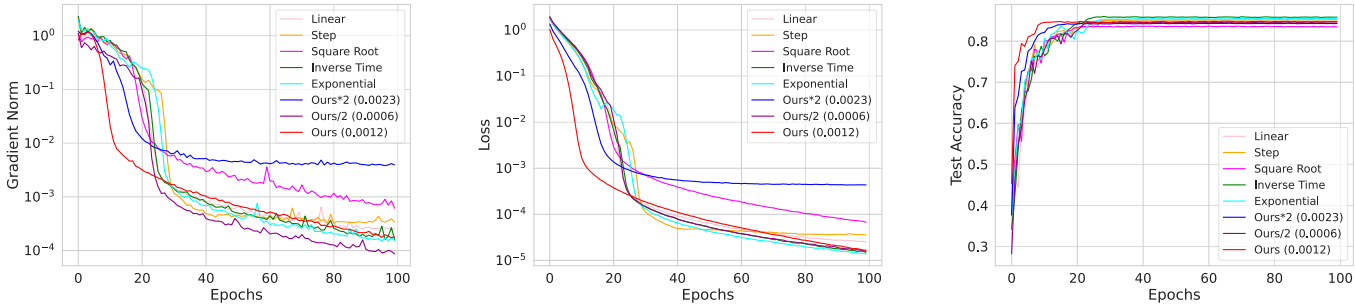


Fig. 5. *Comparison with schedulers: Mini-batch* experiments on VGG-9 architecture with CIFAR-10 data. Left: Grad. norm versus epochs, Middle: Training loss versus epochs, Right: Test acc. versus epochs.

observed that in the mini-batch setting, our stepsize performs the best or on par with other stepsizes and schedulers. It also effectively reduces the gradient norm of loss more aggressively compared to other schedulers and constant stepsizes. This trend aligns with the behavior observed in the full-batch setting across all plots, including gradient norm, training loss, and test accuracy.

D. Experiments on CNNs and ViT

This section presents our main experiments. Training large CNNs and ViT, such as ResNet-18, MobileNet-V2 and ViT-Base, on challenging datasets like ImageNet-100 is complex and time-consuming. The stepsize is a crucial hyperparameter that must be efficiently tuned to achieve high performance. Tuning the stepsize involves training the model across various stepsizes to find the optimal one. Alternatively, one can use stepsize schedulers, but these also have their hyperparameters that need proper tuning. Moreover, stepsize schedulers may suffer from issues mentioned in Section I. To evaluate whether our proposed stepsize performs on par with other stepsizes and schedulers, we conducted experiments on various CNNs: LeNet (5 layers), VGG-9 (9 layers), ResNet-18 (18 layers) and MobileNet-V2 (53 layers), using the CIFAR-10 and ImageNet-100 datasets. All experimental plots that include CIFAR-100 are made available in Sections C-E2, C-E4, C-F2, and C-F4 of the supplementary material. As illustrated in Figs. 4–9 our proposed stepsize scheduler performs well with CNNs, performing on par with various schedulers and constant stepsizes, hence eliminating the need of time-consuming stepsize tuning in complex models also. We can see a gap of at least 10^{-1} order between our stepsize and the best-performing scheduler's gradient norm curve for LeNet experiments. In the case of MobileNet-V2 (Fig. 9), we observe that the square root scheduler diverges, indicating that schedulers also require tuned parameters to perform well. On the other hand, our proposed stepsize can be simply plugged in to achieve comparable results with the best-performing scheduler.

Figs. 6 and 8 show that extremely high (10^{-1}) and extremely low (10^{-5}) stepsizes do not perform well, whereas our stepsize strikes a balance between these extremes and yields the best test accuracy. This section clearly and empirically demonstrates the effectiveness of our proposed stepsize in model training. With sufficient empirical evidence, we suggest that this proposed stepsize can be directly used to train models, thus reducing the time required for stepsize hyperparameter search.

This effectiveness is not limited to just CNN-based models but also extends to transformer-based architectures. In particular, we observe similar performance improvements when applying our stepsize to the ViT-Base, as illustrated in Figs. 10 and 11. When comparing our method with other constant stepsizes, we find that a constant stepsize of 0.1 causes instability, leading to `NaN` values during training and resulting in undefined (NA) accuracy scores, as shown in Table I. Further, as shown in Table II, stepsize schedulers like exponential and inverse time perform very poorly, where the gradient norms and loss values rapidly increase midway through training. In contrast, our proposed stepsize is highly

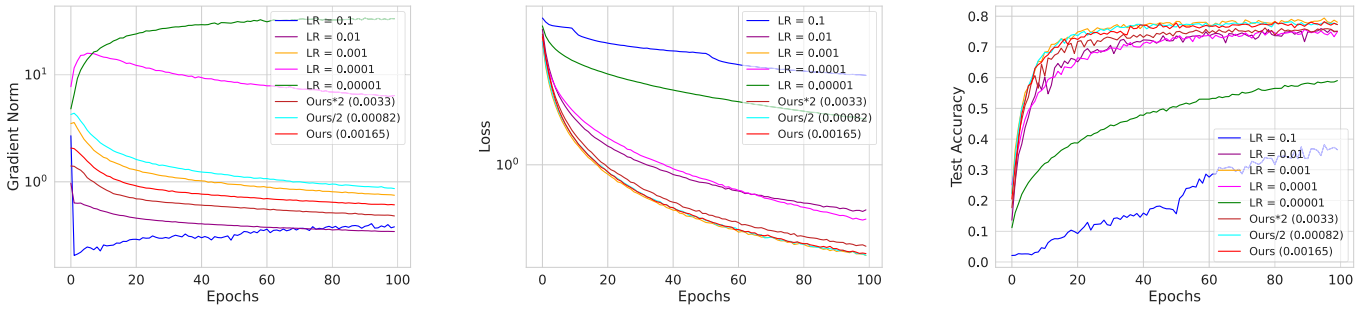


Fig. 6. *Comparison with constant stepsizes: Mini-batch* experiments on ResNet-18 architecture with ImageNet-100 data. Left: Grad. norm versus epochs, Middle: Training loss versus epochs, Right: Test acc. versus epochs.

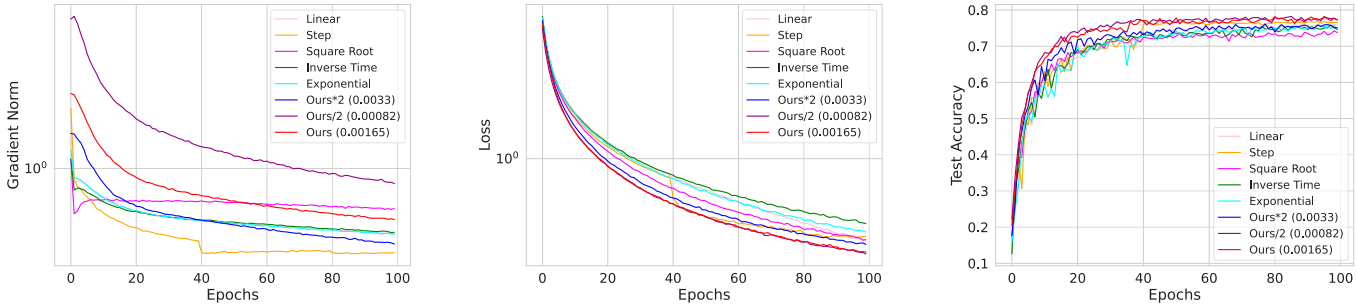


Fig. 7. *Comparison with schedulers: Mini-batch* experiments on ResNet-18 architecture with ImageNet-100 data. Left: Grad. norm versus epochs, Middle: Training loss versus epochs, Right: Test acc. versus epochs.

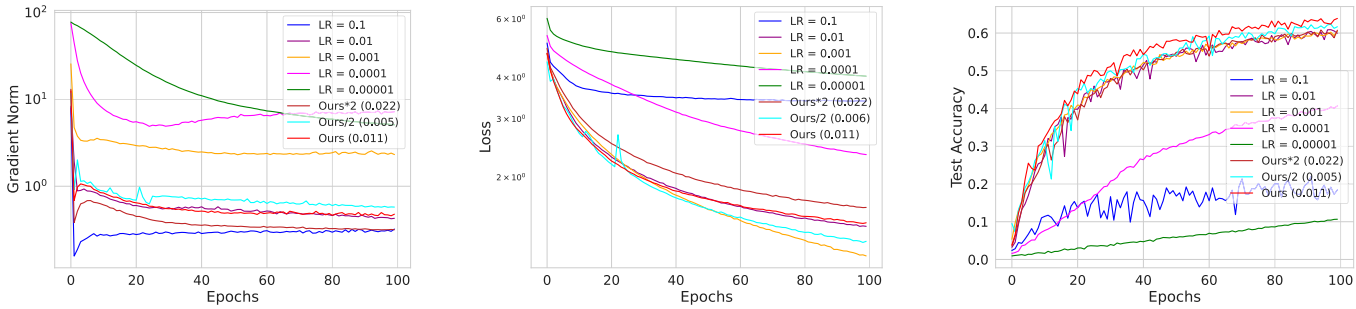


Fig. 8. *Comparison with constant stepsizes: Mini-batch* experiments on MobileNet-V2 architecture with ImageNet-100 data. Left: Grad. norm versus epochs, Middle: Training loss versus epochs, Right: Test acc. versus epochs.

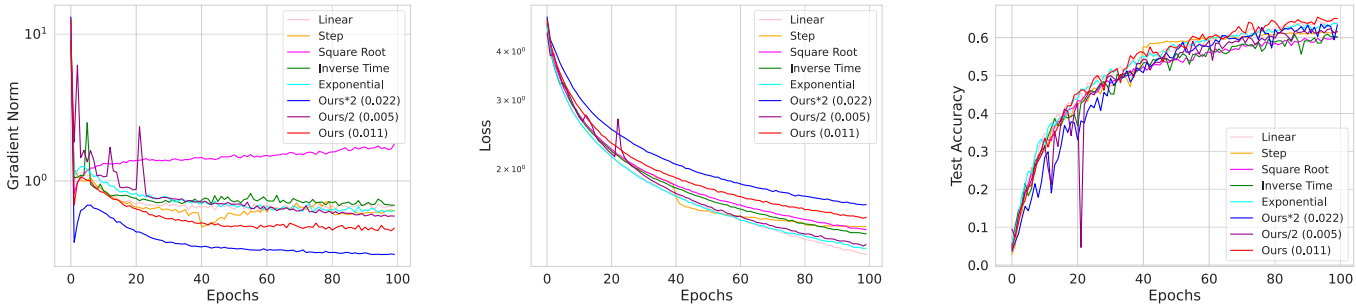


Fig. 9. *Comparison with schedulers: Mini-batch* experiments on MobileNet-V2 architecture with ImageNet-100 data. Left: Grad. norm versus epochs, Middle: Training loss versus epochs, Right: Test acc. versus epochs.

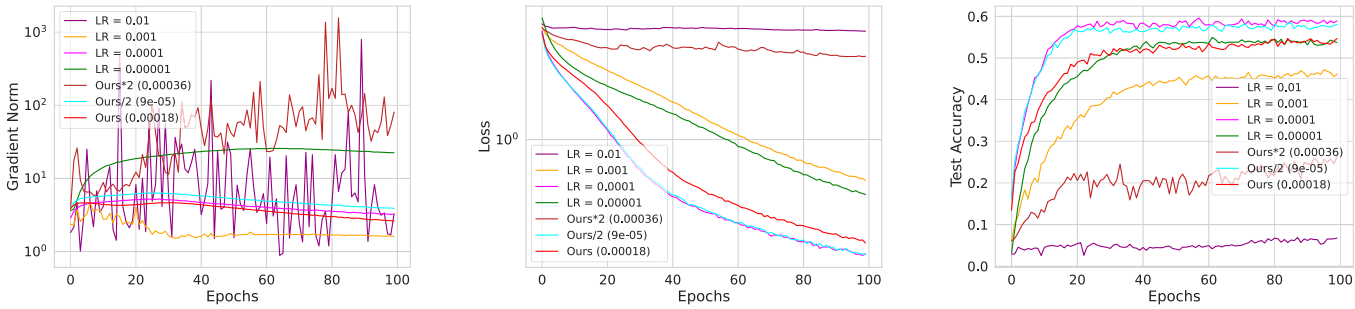


Fig. 10. *Comparison with constant stepsizes: Mini-batch* experiments on ViT architecture with ImageNet-100 data. Left: Grad. norm versus epochs, Middle: Training loss versus epochs, Right: Test acc. versus epochs.

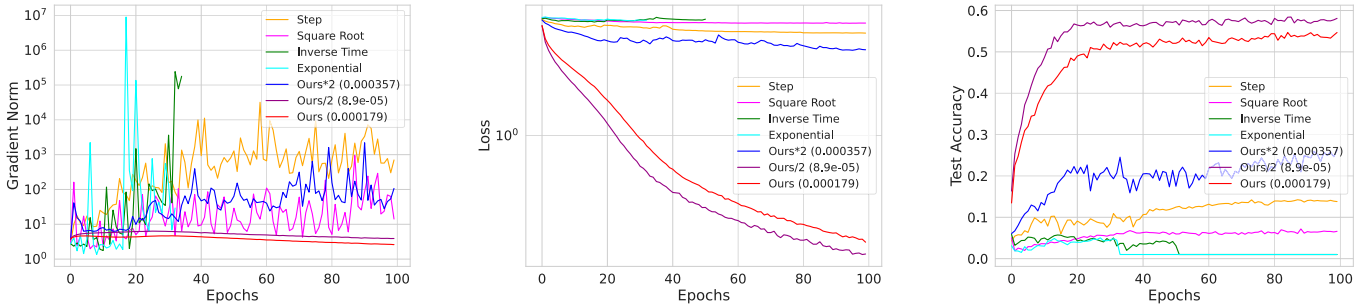


Fig. 11. *Comparison with schedulers: Mini-batch* experiments on ViT architecture with ImageNet-100 data. Left: Grad. norm versus epochs, Middle: Training loss versus epochs, Right: Test acc. versus epochs.

stable, converges more effectively, and achieves approximately three times better test accuracy compared to some of its counterparts.

When compared to constant stepsize strategies, we found that a stepsize of 0.1 resulted in `NaN` values from the very beginning of training, forcing us to exclude it from further experiments. In contrast, our proposed stepsize demonstrates strong stability and produces consistent performance across gradient norm, loss, and test accuracy metrics. While certain fixed stepsizes, such as 0.001, show marginally better behavior in terms of gradient norm, they fail to translate this into improved test accuracy and are outperformed by our method.

Our proposed stepsize not only demonstrates stability and superior performance but also serves as a simple, effective alternative to commonly used decaying stepsize schedulers. Unlike traditional schedulers, which often require careful tuning and can suffer from divergence during training [8], our stepsize is constant and derived from network and data dynamics, significantly reducing the risk of divergence. Empirically, across all our experiments on both CNNs and ViT-Base, our method remained stable and never led to divergent behavior, making it a reliable and plug-and-play replacement for more fragile decaying schedules.

E. Large Scale Experiment on Full ImageNet Dataset

To evaluate the generalization ability of the proposed method in a large-scale regime, we conducted comprehensive image classification experiments on the ImageNet-1K dataset. We primarily benchmarked our approach on ResNet-18, a widely adopted backbone in optimizer studies due to its balanced computational cost and sensitivity to optimization dynamics.

We performed extensive comparisons with a variety of stepsize schedulers, including linear decay, step decay, square-root decay, inverse time decay, exponential decay, and the widely used cosine decay. Additionally, for completeness, we benchmarked our proposed method against a diverse set of optimization algorithms, encompassing both classical and recently proposed approaches, namely AdaBound [19], Yogi [44], MSVAG [45], RAdam [46], and AdamW [47].

These comparisons provide a comprehensive perspective on how the proposed learning-rate formulation interacts with modern optimization strategies under realistic large-scale training conditions. Due to the heavy computational burden, we could not perform an extensive hyperparameter search; instead, we report the result of AdaBelief with the default parameters of Adam ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$) and decoupled weight decay as in [46], [47]; for other optimizers, we report the *best result in the literature*.

As shown in Table III, our proposed combination of Adam with the fixed stepsize variation achieves substantially higher Top-1 accuracy than all other Adam-based learning rate schedulers on ImageNet using ResNet-18. This demonstrates the effectiveness of our stepsize formulation when coupled with Adam. Furthermore, Table IV compares our method with a range of state-of-the-art optimizers. While our approach does not surpass the very best dedicated optimizers such

TABLE I
THE TABLE BELOW COMPARES THE TRAINING AND TESTING ACCURACIES (HIGHER IS BETTER) OF DIFFERENT MODELS AND DATASETS USING VARIOUS CONSTANT STEPSIZES

Model	Dataset	Metric	10^{-1}	10^{-2}	10^{-3}	10^{-4}	10^{-5}	α_{ours}	$2 \times \alpha_{\text{ours}}$	$\alpha_{\text{ours}}/2$
LeNet	CIFAR-10	Train Acc	88.70	100	96.31	67.98	42.56	100	100	99.27
		Test Acc	59.37	62.09	55.63	59.28	42.28	59.19	60.94	58.0
	MNIST	Train Acc	100	100	100	98.2	84.64	100	100	100
		Test Acc	98.98	99.25	99.12	98.1	85.55	99.20	99.27	99.19
VGG-9	CIFAR-10	Train Acc	97.50	100	100	100	92.30	100	100	100
		Test Acc	78.83	85.86	80.88	71.02	67.84	85.40	85.86	81.39
	ImageNet-100	Train Acc	40.92	76.6	77.54	63.87	35.59	77.56	78.00	76.17
		Test Acc	38.60	68.96	70.14	61.08	36.7	70.21	70.39	69.07
ResNet-18	CIFAR-10	Train Acc	98.67	99.70	99.76	100	100	100	99.78	99.41
		Test Acc	69.19	74.68	74.03	49.48	44.77	76.45	75.28	71.82
	ImageNet-100	Train Acc	33.48	83.19	89.42	84.57	57.94	89.26	88.49	89.45
		Test Acc	36.54	75.08	78.04	74.92	59.04	77.28	75.10	77.24
MobileNet-V2	CIFAR-10	Train Acc	86.90	96.98	94.59	70.12	27.71	96.7	94.50	91.40
		Test Acc	63.17	67.21	33.15	18.15	14.28	68.04	67.47	67.79
	ImageNet-100	Train Acc	17.93	62.30	68.33	40.62	08.89	61.39	58.25	65.30
		Test Acc	18.44	59.96	60.56	40.7	10.66	63.75	61.62	62.08
ViT	ImageNet-100	Train Acc	NA	3.50	42.24	46.99	43.67	46.58	13.71	47.03
		Test Acc	NA	6.74	46.12	58.92	53.74	54.68	26.30	58.10

Note: It is evident that the variations of our proposed stepsize perform consistently better or on par with the other stepsizes. The first, second and third best performing stepsizes are colored in red, green and brown respectively.

TABLE II
THE TABLE BELOW COMPARES THE TRAINING AND TESTING ACCURACIES (HIGHER IS BETTER) OF DIFFERENT MODELS AND DATASETS USING VARIOUS SCHEDULERS

Model	Dataset	Metric	Linear	Exponent	Step	Sq. root	Inverse	α_{ours}	$2 \times \alpha_{\text{ours}}$	$\alpha_{\text{ours}}/2$
LeNet	CIFAR-10	Train Acc	100	100	100	99.65	100	100	100	99.21
		Test Acc	61.22	57.66	61.32	59.50	60.97	61.28	61.08	57.71
	MNIST	Train Acc	100	100	100	100	100	100	100	100
		Test Acc	99.71	99.70	99.74	99.68	99.71	99.8	99.76	99.77
VGG-9	CIFAR-10	Train Acc	100	100	100	100	100	100	100	100
		Test Acc	85.16	85.58	85.24	84.49	85.86	84.94	84.87	84.91
	ImageNet-100	Train Acc	80.60	71.13	78.03	80.34	76.88	77.43	78.01	76.95
		Test Acc	71.94	68.26	71.86	70.82	70.09	70.86	70.11	68.30
ResNet-18	CIFAR-10	Train Acc	100	100	100	100	100	100	99.78	99.41
		Test Acc	75.46	75.84	76.17	72.81	75.75	76.45	75.28	71.82
	ImageNet-100	Train Acc	88.12	86.93	87.71	87.93	85.72	89.26	88.49	89.45
		Test Acc	75.90	74.78	76.50	73.76	74.56	77.28	75.10	77.24
MobileNet-V2	CIFAR-10	Train Acc	98.97	66.10	100	98.92	97.58	97.64	95.71	93.10
		Test Acc	69.35	69.60	65.42	53.12	63.28	69.48	69.38	69.49
	ImageNet-100	Train Acc	66.72	66.01	63.81	63.88	64.07	66.10	64.40	65.56
		Test Acc	64.70	64.66	62.41	60.11	60.14	66.85	64.57	62.44
ViT	ImageNet-100	Train Acc	1.14	1.00	8.34	3.34	1.12	46.23	13.61	46.68
		Test Acc	1.65	1.71	13.22	8.70	1.70	54.22	27.50	59.14

Note: It is evident that the variations of our proposed stepsize perform consistently better or on par with the other schedulers. The first, second and third best performing stepsizes/schedulers are colored in red, green and brown respectively.

TABLE III
TOP-1 ACCURACY (%) OF DIFFERENT LEARNING RATE SCHEDULERS WITH ADAM ON IMAGENET USING RESNET-18

Scheduler	Step	Linear	Cosine	Exp.	Inv. Time	Sqrt	α_{ours} (5.9×10^{-4})	$2\times\alpha_{\text{ours}}$	$\alpha_{\text{ours}}/2$
Top-1 Acc. (%)	57.33	62.77	62.53	62.59	60.59	62.28	67.53	66.71	67.79

Note: The first, second and third best performing stepsizes are colored in red, green and brown respectively.

TABLE IV
TOP-1 ACCURACY (%) WITH VARIOUS OPTIMIZERS ON IMAGENET USING RESNET-18

Optimizer	AdaBelief	AdaBound†	Yogi†	Adam†	MSVAG	RAdam‡	AdamW†	Adam + α_{ours} (5.9×10^{-4})	Adam + $2\times\alpha_{\text{ours}}$	Adam + $\alpha_{\text{ours}}/2$
Top-1 Acc. (%)	70.08	68.13	68.23	63.79	65.99	67.62	67.93	67.53	66.71	67.79

Note: † is reported in [52], ‡ is reported in [46]. The first, second and third best performing stepsizes are colored in red, green and brown respectively.

as AdaBelief and Yogi, it achieves performance that is competitive with the famous go-to optimizer AdamW, despite its conceptual simplicity and minimal modification to Adam. It also beats other optimizers like MSVAG and RAdam. These results highlight the generality and effectiveness of our proposed method in a large-scale experimental setting.

F. Experiments on Large Language Models

To test the generalizability of our proposed learning-rate method beyond image classification, we fine-tuned two instruction-tuned LLMs, LLaMA-3.2-3B-Instruct [48], and Qwen-3-4B-Instruct [49], on two distinct NLP tasks: text classification (AG News) and question answering/generation

TABLE V
ANALYTICALLY DERIVED STEPSIZES (α_{ours}) USED IN LLM LORA FINE-TUNING EXPERIMENTS

Model	Ag News	SQuAD
LLaMA-3.2-3B-Instruct	3.2×10^{-4}	3.7×10^{-4}
Qwen-3-4B-Instruct	3.8×10^{-4}	5.7×10^{-4}

TABLE VI
DATASET: SQUAD, OPTIMIZER: ADAM, TASK: QA (GENERATION)

	Step	Linear	Cosine	Exponential	Sqrt	Inv-time	α_{ours}	$2 \times \alpha_{\text{ours}}$	$\alpha_{\text{ours}}/2$
Exact Match									
LLaMA-3.2-3B-Instruct	0.7061	0.7034	0.7096	0.7013	0.7032	0.7046	0.6998	0.6941	0.7141
Qwen-3-4B-Instruct	0.7173	0.7153	0.7189	0.7108	0.7123	0.7131	0.7135	0.6965	0.7176
F1									
LLaMA-3.2-3B-Instruct	0.8546	0.8531	0.8572	0.8518	0.8523	0.8529	0.8508	0.8408	0.8585
Qwen-3-4B-Instruct	0.8661	0.8642	0.8653	0.8621	0.8630	0.8629	0.8575	0.8577	0.8660

Note: The first, second and third best performing stepsizes are colored in red, green and brown respectively.

TABLE VII
DATASET: AG NEWS, OPTIMIZER: ADAM

	Step	Linear	Cosine	Exponential	Sqrt	Inv-time	α_{ours}	$2\times \alpha_{\text{ours}}$	$\alpha_{\text{ours}}/2$
Accuracy									
LLaMA-3.2-3B-Instruct	0.9164	0.9154	0.9169	0.9145	0.9162	0.9161	0.9085	0.8959	0.9166
Qwen-3-4B-Instruct	0.9128	0.9127	0.9130	0.9126	0.9124	0.9126	0.9117	0.8813	0.9132
F1									
LLaMA-3.2-3B-Instruct	0.9161	0.9159	0.9163	0.9152	0.9153	0.9157	0.9077	0.8940	0.9162
Qwen-3-4B-Instruct	0.9132	0.9125	0.9130	0.9122	0.9126	0.9123	0.9108	0.8907	0.9134

Note: The first, second and third best performing stepsizes are colored in red, green and brown respectively.

TABLE VIII
DATASET: SQUAD, SCHEDULER: COSINE, TASK: QA (GENERATION)

	AdaBelief	AdaBound	AdamW	Yogi	RAdam	Adam + α_{ours}	Adam + $2 \times \alpha_{\text{ours}}$	Adam + $\alpha_{\text{ours}}/2$
Exact Match								
LLaMA-3.2-3B-Instruct	0.7122	0.6836	0.7136	0.7088	0.7202	0.6998	0.6941	0.7141
Qwen-3-4B-Instruct	0.7275	0.6992	0.7333	0.7170	0.7322	0.7135	0.6965	0.7176
F1								
LLaMA-3.2-3B-Instruct	0.8605	0.8345	0.8636	0.8529	0.8648	0.8508	0.8408	0.8585
Qwen-3-4B-Instruct	0.8701	0.8487	0.8728	0.8612	0.8724	0.8575	0.8577	0.8660

Note: The first, second and third best performing stepsizes are colored in red, green and brown respectively.

TABLE IX
DATASET: AG NEWS, SCHEDULER: COSINE

	AdaBelief	AdaBound	AdamW	Yogi	RAdam	Adam + α_{ours}	Adam + $2\times \alpha_{\text{ours}}$	Adam + $\alpha_{\text{ours}}/2$
Accuracy								
LLaMA-3.2-3B-Instruct	0.9343	0.8913	0.9355	0.9154	0.9307	0.9085	0.8959	0.9166
Qwen-3-4B-Instruct	0.9330	0.8823	0.9328	0.9098	0.9272	0.9117	0.8813	0.9132
F1								
LLaMA-3.2-3B-Instruct	0.9341	0.8910	0.9361	0.9181	0.9308	0.9077	0.8940	0.9162
Qwen-3-4B-Instruct	0.9324	0.8822	0.9332	0.9097	0.9267	0.9108	0.8907	0.9134

Note: The first, second and third best performing stepsizes are colored in red, green and brown respectively.

(SQuAD [50]). Fine tuning employed LoRA [53] adapters inserted into the attention layers. We compared our learning-rate strategy (with Adam) against multiple learning-rate schedulers and additionally evaluated a range of modern optimizers paired with a cosine decay schedule, as described in the Section V-E. This setup was designed to probe transferability across model families and task types while isolating the impact of optimizer and scheduler choices on downstream performance.

We included the LLM experiments solely for completeness, rather than as a full-scale study. The LoRA adapter fine-tuning runs are meant to probe transferability to a very different model family (transformer based) and task type (generation), not to exhaustively tune or benchmark every LLM setting. More broadly, our goal with those runs, just like the toy, CNN and large scale ImageNet experiments was to show the proposed stepsize is stable and competitive across a range of architectures and datasets (vision $\rightarrow$ language) so readers can see the method's behavior in diverse regimes; given compute constraints we deliberately kept the LLM batch size and runs small and controlled so they serve as a robustness check rather than a definitive LLM benchmark.

For AG News we used a fixed split of 50 000 training and 7600 evaluation examples; for SQuAD we used a fixed split of 50 000 training and 10 600 evaluation examples. LoRA adapters were applied to the attention layers with rank $r = 16$, $\alpha = 32$, and dropout $= 0.05$. Each LLM is fine-tuned for 3125 steps (batch size $= 16$, epoch $= 1$). The learning rates for the LLM LoRA fine-tuning experiments are mentioned in Table V.

From Tables VI and VII, our proposed fixed stepsize with Adam consistently achieves superior results compared to all other stepsize schedulers across both SQuAD and AG News datasets. On SQuAD, which evaluates question answering and generation capabilities, the method yields the highest Exact Match and F1 scores among all Adam-based schedulers, indicating improved optimization and training. Similar trends are observed on AG News, where our stepsize formulation marginally but consistently outperforms other schedulers.

When compared against modern adaptive optimizers like AdaBound [19], Yogi [44], MSVAG [45], RAdam [46], and AdamW [47], under the cosine scheduler (Tables VIII and IX), our Adam + stepsize combination achieves competitive performance, closely matching strong baselines such as AdaBelief, AdamW, and RAdam. It beats optimizers like Yogi and AdaBound consistently across LLM models and datasets. Although the proposed approach does not exceed the top-performing optimizers in all cases, its simplicity and strong cross-domain performance, spanning both vision and language tasks, demonstrate its practical effectiveness and generalizability.

VI. OBSERVATIONS AND DISCUSSION

The analytic stepsize α_{ours} was derived from first principles and depends on quantities such as the initial loss $\mathcal{L}(\mathbf{w}_0)$, the estimated Lipschitz constant $\hat{K}$, and the training horizon T, all of which are estimated in practice. Consequently, the $\times 2$ and $/2$ variants serve as natural robustness checks to validate the stability of our theoretical stepsize under mild estimation uncertainty. Across all our experiments on CNNs (Section V-D), ViTs (Section V-D), and large-scale ImageNet training (Section V-E), α_{ours} consistently delivers strong performance (refer to Tables I–III), matching or outperforming baselines. The only regime where the halved variant $\alpha_{\text{ours}}/2$ yields superior results is in the LLM fine-tuning setting (Section V-F: Tables VI and

VII). This deviation can be attributed to the distinct characteristics of LoRA based fine-tuning, where only adapter parameters are updated, effectively altering the parameter space and the curvature/Hessian profile compared to full-model training. For the LLM LoRA fine-tuning protocol, the practical training conditions differ substantially from those in our full-network experiments. In this setup, only the LoRA adapter parameters are updated. Accordingly, the Lipschitz constant $\hat{K}$ in our formulation is estimated with respect to the LoRA weights only. However, the effective curvature experienced by these parameters is indirectly influenced by the frozen backbone, since the adapter updates propagate through fixed nonlinear transformations defined by the base model. This results in a highly anisotropic and locally varying curvature landscape, where the gradient magnitudes and smoothness assumptions underlying our theoretical stepsize derivation hold only approximately. In addition, due to compute resource constraints, the batch size was limited to 16, introducing higher stochastic gradient variance, and the fine-tuning horizon was short (about 3 K steps), both of which further increase sensitivity to stepsize magnitude. Under these combined factors, the analytically derived α_{ours} can be slightly aggressive, while halving it ($\alpha_{\text{ours}}/2$) yields smoother updates and improved downstream QA and classification performance. In contrast, across CNN, ViT, and large scale ImageNet experiments where full model optimization and larger batch sizes were used, α_{ours} consistently performs best, making the LLM LoRA setup the only regime where the halved variant is preferable.

VII. Conclusion and Limitations

Our experimental results show that the proposed stepsize can be directly used for model training, eliminating the need for extensive tuning while maintaining high accuracy. It consistently matches or outperforms constant stepsizes and decaying schedulers across various architectures, including fully connected networks, LeNet, VGG-9, ResNet-18, and MobileNet-V2, trained on datasets such as MNIST, CIFAR-10, CIFAR-100, and ImageNet-100. We have further extended our evaluation to large scale settings, including full ImageNet training and instruction tuned LLMs, demonstrating that the same stepsize remains stable and competitive without any hyperparameter adjustments. The stepsize avoids divergence issues observed with some schedulers and extreme stepsizes, offering a reliable and simple solution for deep learning training. This makes it a practical, time-saving choice for practitioners, ensuring robust performance without extensive tuning. *Limitations and Future Work*: 1) We restricted our experiments to only cross-entropy loss. In the future, we will address various types of loss functions. 2) It is still open to tightening the convergence rate of Adam.

References

[1] L. Bottou, "Stochastic gradient descent tricks," in *Neural Networks: Tricks of the Trade.* Berlin, Germany: Springer, 2012.

[2] L. Bottou, "Curiously fast convergence of some stochastic gradient descent algorithms," in *Proc. Symp. Learn. Data Sci.*, 2009, pp. 2624–2633.

[3] J. Duchi, E. Hazan, and Y. Singer, "Adaptive subgradient methods for online learning and stochastic optimization," *J. Mach. Learn. Res.*, vol. 12, no. 7, 2011.

[4] T. Tieleman and G. Hinton, "RMSProp: Divide the gradient by a running average of its recent magnitude. Coursera: Neural networks for machine learning," in *Proc. COURSERA Neural Netw. Mach. Learn*, vol. 17, 2012.

[5] D. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent.*," 2015, pp. 1–13.

[6] M. D. Zeiler, "Adadelta: An adaptive learning rate method," 2012, *arXiv:1212.5701.*

[7] S. Vaswani, F. Bach, and M. Schmidt, "Fast and faster convergence of SGD for over-parameterized models and an accelerated perceptron," in *Proc. 22nd AISTATS.* PMLR, Okinawa, Japan, 2019, pp. 1195–1204.

[8] J. R. Sashank, K. Satyen, and K. Sanjiv, "On the convergence of Adam and beyond," in *Proc. ICLR*, 2018.

[9] J. Martens and R. Grosse, "Optimizing neural networks with Kronecker-factored approximate curvature," in *Proc. Int. Conf. Mach. Learn.*, 2015.

[10] S. De, A. Yadav, D. Jacobs, and T. Goldstein, "Automated inference with adaptive batches," in *Proc. Artif. Intell. Statist.*, 2017.

[11] R. Babanezhad Harikandeh, M. O. Ahmed, A. Virani, M. Schmidt, J. Konečnỳ, and S. Sallinen, "Stopwasting my gradients: Practical SVRG," in *Advances in Neural Information Processing System*, vol. 28, 2015, pp. 2251–2259.

[12] R. Johnson and T. Zhang, "Accelerating stochastic gradient descent using predictive variance reduction," in *Advances in Neural Information Processing System*, vol. 26, 2013, pp. 315–323.

[13] A. Defazio, F. Bach, and S. Lacoste-Julien, "Saga: A fast incremental gradient method with support for non-strongly convex composite objectives," *Advances in Neural Information Processing System*, 2014.

[14] J. Bernstein, Y.-X. Wang, K. Azizzadenesheli, and A. Anandkumar, "signSGD: Compressed optimisation for non-convex problems," in *Proc. Int. Conf. Mach. Learn.* PMLR, Vienna, Austria, 2018, pp. 560–569.

[15] A. Barakat and P. Bianchi, "Convergence rates of a momentum algorithm with bounded adaptive step size for nonconvex optimization," in *Proc. 12th Asian Conf. Mach. Learn., Ser. Proc. Mach. Learn. Res.*, S. J. Pan and M. Sugiyama, Eds. PMLR, 2020, vol. 129, pp. 225–240.

[16] N. Shi, D. Li, M. Hong, and R. Sun, "RMSProp converges with proper hyper-parameter," in *Proc. Int. Conf. Learn. Represent.*, 2020.

[17] R. Tian and A. P. Parikh, "Amos: An Adam-style optimizer with adaptive weight decay towards model-oriented scale," 2022, *arXiv:2210.11693.*

[18] X. Chen, S. Liu, R. Sun, and M. Hong, "On the convergence of a class of Adam-type algorithms for non-convex optimization," 2018, *arXiv:1808.02941.*

[19] L. Luo, Y. Xiong, Y. Liu, and X. Sun, "Adaptive gradient methods with dynamic bound of learning rate," 2019, *arXiv:1902.09843.*

[20] A. Défossez, L. Bottou, F. Bach, and N. Usunier, "A simple convergence proof of Adam and Adagrad," 2020, *arXiv:2003.02395.*

[21] F. Zou, L. Shen, Z. Jie, W. Zhang, and W. Liu, "A sufficient condition for convergences of Adam and RMSProp," in *Proc. IEEE/CVF CVPR*, 2019.

[22] Z. Zhou, Q. Zhang, G. Lu, H. Wang, W. Zhang, and Y. Yu, "AdaShift: Decorrelation and convergence of adaptive learning rate methods," 2018, *arXiv:1810.00143.*

[23] C. Chen, L. Shen, F. Zou, and W. Liu, "Towards practical Adam: Non-convexity, convergence theory, and mini-batch acceleration," *J. Mach. Learn. Res.*, vol. 23, no. 1, pp. 10411–10457, 2022.

[24] L. Jing, J. Zbontar, and Y. LeCun, "Implicit rank-minimizing autoencoder," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 14736–14746, 2020.

[25] Y. Li, P. Hu, Z. Liu, D. Peng, J. T. Zhou, and X. Peng, "Contrastive clustering," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 8547–8555.

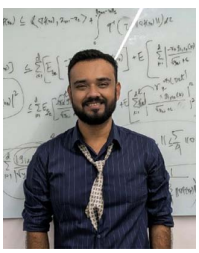
[26] S. De, A. Mukherjee, and E. Ullah, "Convergence guarantees for RMSProp and ADAM in non-convex optimization and an empirical comparison to Nesterov acceleration," 2018, *arXiv:1807.06766.*

[27] M. Li, T. Zhang, Y. Chen, and A. J. Smola, "Efficient mini-batch training for stochastic optimization," in *Proc. 20th ACM SIGKDD*, 2014, pp. 661–670.

[28] A. Virmaux and K. Scaman, "Lipschitz regularity of deep neural networks: Analysis and efficient estimation," in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., New York, NY, USA: Curran Associates Inc, 2018.

[29] M. Fazlyab, A. Robey, H. Hassani, M. Morari, and G. Pappas, "Efficient and accurate estimation of Lipschitz constants for deep neural networks," in *Advances in Neural Information Processing Systems*, H. Wallach, H.

Larochelle, A. Beygelzimer, F. d' Alché-Buc, E. Fox, and R. Garnett, Eds., New York, NY, USA: Curran Associates Inc, 2019.
[30] F. Latorre, P. Rolland, and V. Cevher, "Lipschitz constant estimation of neural networks via sparse polynomial optimization," 2020, *arXiv:2004.08688.*
[31] M. Fazlyab, A. Robey, H. Hassani, M. Morari, and G. Pappas, "Efficient and accurate estimation of Lipschitz constants for deep neural networks," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
[32] H. Gouk, E. Frank, B. Pfahringer, and M. J. Cree, "Regularisation of neural networks by enforcing Lipschitz continuity," *Mach. Learn.*, vol. 110, pp. 393–416, 2021.
[33] Federer, H. *Geometric Measure Theory.* Berlin, Germany: Springer, 2014.
[34] H. Müntz et al., "Solution directe de l'équation séculaire et de quelques problemes analogues transcendants," *CR Acad. Sci. Paris*, vol. 156, pp. 43–46, 1913.
[35] S. Boyd and L. Vandenberghe, *Convex Optimization.* Cambridge, U.K.: Cambridge Univ. Press, 2004.
[36] P. Wolfe, "Convergence conditions for ascent methods," *SIAM Rev.*, vol. 11, no. 2, pp. 226–235, 1969.
[37] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2015, pp. 1026–1034.
[38] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
[39] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556.*
[40] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recogn.*, 2016, pp. 770–778.
[41] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recogn.*, 2018, pp. 4510–4520.
[42] A. Dosovitskiy, et al., "An image is worth 16x16 words: Transformers for image recognition at scale," 2020, *arXiv:2010.11929*.
[43] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis.Pattern Recogn.*, Piscataway, NJ, USA: IEEE Press, 2009, pp. 248–255.
[44] M. Zaheer, S. Reddi, D. Sachan, S. Kale, and S. Kumar, "Adaptive methods for nonconvex optimization," in *Advances in Neural Information Processing Systems*, vol. 31, 2018, pp. 9815–9825.
[45] L. Balles and P. Hennig, "Dissecting Adam: The sign, magnitude and variance of stochastic gradients," in *Proc. Int. Conf. Mach. Learn.* PMLR, 2018, pp. 404–413.
[46] L. Liu, et al., "On the variance of the adaptive learning rate and beyond," 2019, *arXiv:1908.03265.*
[47] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2017, *arXiv:1711.05101.*
[48] A. Dubey, et al., "The Llama 3 herd of models," 2024, *arXiv:2407.21783*.
[49] A. Yang, et al., "Qwen3 technical report," 2025, *arXiv:2505.09388.*
[50] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "SQuAD: 100,000+ questions for machine comprehension of text," 2016, *arXiv:1606.05250.*
[51] G. E. Dahl, et al., "Benchmarking neural network training algorithms," 2023, *arXiv:2306.07179.*
[52] J. Chen, D. Zhou, Y. Tang, Z. Yang, Y. Cao, and Q. Gu, "Closing the generalization gap of adaptive gradient methods in training deep neural networks," 2018, *arXiv:1806.06763.*
[53] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *ICLR*, vol. 1, no. 2, p. 3, 2022.



Alokendu Mazumder received the M.Tech. degree in computer science and engineering, from the Indian Institute of Technology (IIT) Jammu, India, in 2021, where he worked on the problem of anomaly detection over dynamic graphs in collaboration with Ericsson Research. He is currently working toward the Ph.D. degree in theoretical deep learning and computer vision with the Indian Institute of Science (IISc), Bengaluru, India.

His research interests include stochastic optimization, deep learning theory, and computer vision.

Dr. Mazumder is a recipient of the prestigious Prime Minister's Research Fellowship (PMRF) (2022 cycle), awarded by the Ministry of Education, Government of India. He was also awarded the MIT International Science & Technology Initiative (MIT-MISTI) grant and the Kotak AI-ML Ph.D. Fellowship in 2025. He has held research internships at IBM Research India, Dolby Laboratories, and Ericsson Research.

Rishab Sabharwal is currently working toward the master's degree in artificial intelligence with the University of Edinburgh, Edinburgh, Scotland.

Prior to this, he worked as a Predoctoral Researcher with the IISc, Bengaluru, India, where he collaborated with the National Payments Corporation of India and contributed to several research projects in computer vision and graph neural networks. His research interests include interpretability and AI safety, with a focus on monitoring capabilities using chain-of-thought reasoning and exploring cross-representational guardrails for safe large language models.

Bhartendu Kumar received the master's degree in artificial intelligence from the IISc, Bengaluru, India, in 2023, where he worked on convergence guarantees in learning algorithms.

He was a Researcher with TCS Research, detecting and mitigating biases in deep learning solutions. He is currently a Researcher with the Microsoft Research to adapt and extend Agentic AI to software engineering pipeline, especially AIOps. The work transcends from reasoning, convergence guarantees and collaborations protocols in multi-agent system.

Manan Tayal (Graduate Student Member, IEEE) received the B. Tech. degree in mechanical engineering from the IIT, Bombay, India, in 2021. He is currently working toward the Ph.D. degree with Robert Bosch Center for Cyber Physical Systems, IISc, Bengaluru, India.

His research interests include intersection of control, optimization, and machine learning, with a focus on safety enforced decision-making in dynamical systems.

Mr. Tayal is a recipient of Prime Minister's Research Fellowship (PMRF), Government of India.

Chirag Garg is currently working toward the B.Tech. degree with the National Institute of Technology, Raipur, India, where he is interested in understanding how neural networks learn and use internal representations.

His work focuses on representation learning and latent space structure in deep models. He has interned at the CiSTUP department, IISc, Bengaluru, India, where he worked on deterministic autoencoders with hyperspherical latent spaces. His research interests include practical and empirical aspects of building better and more interpretable learning systems.

Arnab Roy is currently working toward the M.Tech. degree in artificial intelligence with the IISc, Bengaluru, India.

During his tenure at IISc, he worked as a summer intern with PrivaSapien, contributing to Responsible AI initiatives, including safety and privacy-aware large language model systems. Prior to IISc, he worked with the Indian Oil Corporation Limited (IOCL), where he led data-driven initiatives in predictive maintenance, anomaly detection, and industrial analytics. His academic interests are strongly grounded in applied mathematics and statistics, motivating his transition toward advanced research in AI. His research interests include deep learning, graph theory, and computer vision, with a particular focus on novel view synthesis and 3-D Gaussian Splatting.

Punit Rathore (Member, IEEE) received the Ph.D. degree in electronics and electrical engineering from the Department of Electrical and Electronics Engineering, University of Melbourne, Australia, in 2019.

From 2011 to 2014, he was a Researcher with the Automation Division, TATA Steel Limited, India, where he developed several image processing and machine vision-based real-time systems for manufacturing industries. He worked as a Postdoctoral Researcher with the Senseable City Lab, Massachusetts Institute of Technology (MIT), Cambridge, MA, USA, for two years, and with the Grab-NUS AI Lab, Institute of Data Science, National University of Singapore (NUS), Singapore, for nine months. He is currently an Assistant Professor with the IISc, Bengaluru, India, and the Robert Bosch Centre for Cyberphysical Systems (RBCCPS), Bengaluru, India, jointly affiliated with the Centre for Infrastructure, Sustainable Transportation, and Urban Planning (CiSTUP). His research interests include big data analytics, unsupervised, semi-supervised, and self-supervised learning, visual analytics using deep learning, explainable AI/ML, data-driven analytics for autonomous systems and transportation, spatio-temporal data mining, and streaming data analytics. He is also interested in applying machine learning and AI to solve complex real-world problems in areas such as transportation, agriculture, healthcare, infrastructure, robotics, and computer vision.